SC2006 Lab 3 Supplementary Report: AI-Based Project Initialisation

URECAMatch, Lab Group SCEF, Group 4

Team: Ian Tan, Robin Teoh, Nicholas Ng, Edwin Tan Qi Lun, Benjamin Ng

AI used: Claude Opus 5.5 (Claude Code Agent, VS Code)

1. Overview

An agentic AI tool was used to produce two different aids for prototyping URECAMatch: **i)** an updated technology stack for the current iteration of the SRS, and **ii)** a minimal code skeleton template. As with previous AI reports, the team opted to use the Claude Opus 5.5 model by Anthropic as it offered more robust in-IDE integration features.

2. Technology Stack

The goal of this section is to generate a technology stack suggestion suitable for implementing URECAMatch, revised against the initial suggestions generated in Lab 2.

2.1 Technology Stack: Methodology

Similar to the previous iteration of the AI report, a prompt summarizing the team's key competencies as well as a short write-up of URECAMatch was submitted to the agentic AI tool. The two documents attached to the prompt were updated for this iteration, namely:

a. An updated class diagram (See: SRS Appendix)
b. An updated pared-down version of the SRS containing various key sections such as the FRs (See: SRS Section 4), NFRs (See: SRS Section 5), Use Case Descriptions (See: SRS Appendix)

These were all submitted in a new chat session. As per Lab 2, the full text prompt is as follows:

```
Our team is building URECAMatch, a web-based matching portal for NTU's
URECA program. The system has three roles: Students, Project Supervisors,
and Admins. Attached is a proposed class diagram for the software
architecture, as well as a copy of the in-progress Software Requirement
Specifications (SRS).

Summary of Core Features:
* Account and application cycle management.
* Preference ranking submission for both students and project supervisors.
* An automated matching system that finds the best fit for students and
project supervisors based on their ranking submissions.

Based on the uploaded design materials, recommend a technology stack
appropriate for an entry-level software engineering team, considering our
collective expertise in Python, Java, C/C++, TypeScript, Next.js, Supabase,
and Vercel.
```

After the first reply by the AI, a short follow-up prompt was then submitted to compare against the technology stack that was recommended for Lab 2. The follow-up prompt is shown below.

```
Attached is a short .txt document containing the tech stack previously
suggested for URECAMatch. Compare this to what was just suggested and
justify your reasons
```

2.2 Technology Stack: Model Response / Results

The updated suggestion from the AI can be summarized as follows: Next.js and TypeScript was suggested for the web client; one Python FastAPI service with a router per controller; Supabase for PostgreSQL, authentication and file storage; Vercel Cron for scheduled work; and a transactional e-mail API.

When compared against the tech stack from Lab 2, the response furthermore advised against using multiple microservices, Kubernetes, Redis, NTU single sign-on and a hosted ML endpoint, citing “over-engineering far past the required scope” as a reason for the modifications to the technology stack.

The differences between the two AI-suggested technology stacks are shown below in Table 1.

Layer	Lab 2	Lab 3
Back-end	Node.js and FastAPI microservices	One FastAPI service
Data	PostgreSQL plus Redis	Supabase PostgreSQL behind a Store
Auth and files	NTU SSO; AWS S3	Supabase Auth and Storage
Hosting	Docker, Kubernetes on AWS	Vercel and Supabase
Additional features	SentenceTransformers on SageMaker (for AI suggestions)	Resend (for email push notifications)

Table 1. Differences between the AI-suggested technology stacks for Lab 2 and Lab 3 (current).

2.3 Technology Stack: Discussion

As can be seen from Table 1, major aspects of the technology stack have been updated from the suggestion given in Lab 2. While the front-end remains largely the same (*sans* Redux), the back-end, data, authentication, scheduling and hosting stack layers were all revised.

With reference to the updated class diagram (See: SRS Appendix), it can be clearly observed how the attached class diagram influenced the suggested technology stack:

a. Since all controllers share a single Store, the AI proposed one FastAPI service rather than several microservices
b. The Store Interface led to PostgreSQL sitting behind a repository
c. Time-based classes (WindowService, MatchingController.tick) led to Vercel Cron for scheduled work
d. NotificationService and EmailService led to a transactional e-mail API (Resend),
e. StorageService.signedUrl led to private file storage accessed through signed URLs
f. Removing passwordHash from the user class led to the AI-suggested delegation of authentication to Supabase Auth
g. The absence of recommendation or cache class led to dropping SageMaker and Redis

One major reason for the observed differences could also be attributed to the fact that the attached SRS document in the prompt for Lab 3 was updated and hence more well-defined, which led to the AI scaling down the technology stack from an industry-scale AWS platform to a smaller, more tailored stack suitable for the requirements of URECAMatch.

Several aspects of the technology stack suggested in Lab 2 were ultimately adopted for URECAMatch. For instance, the current build of URECAMatch still uses Python and TypeScript for its core implementing languages, along with frameworks such as FastAPI. Lab 3 then replaced the industry-scale AWS platform with a lighter stack built around a single FastAPI service, Supabase and Vercel, as summarized in Table 1.

Upon a manual review of the technology stack suggestions, the group decided to add a new layer dedicated to **Testing and Continuous Integration (CI)**. This would utilize pytest, ruff, ESLint, and GitHub Actions.

Layer	Technology Stack
Front-end	Next.js, TypeScript, Tailwind CSS
Back-end	FastAPI, Python 3.12, Pydantic, uv
Data	Supabase PostgreSQL behind a Store
Auth and files	Supabase Auth, Supabase Storage
Hosting	Vercel, Supabase, Vercel Cron
Notifications	Resend (for email push notifications)
Testing/CI	pytest, ruff, ESLint, tsc, GitHub Actions

Table 2. The final technology stack adopted for URECAMatch. It includes additional details added upon group review, as well as a new layer: Testing/CI layer

3. Minimal Skeleton Template

The goal of this section is to generate a full application skeleton consisting of named empty files (or files with only comments) and sub-directories. This will serve as the initial architecture from which the team will begin the implementation of URECAMatch.

3.1 Minimal Skeleton Template: Methodology

The finalized technology stack in Section 2 of this AI Report (See: Table 2) was then used to form part of the prompt for generating an initial code skeleton. The team defined 2 key rules in the prompt, namely:

1. files should contain only comments (no code of any kind),
2. folders should follow the hierarchy as defined in the attached Class Diagram, Dialog Map, and Boundary Diagram

The prompt used is as follows:

```
Using the attached class diagram, create a minimal project skeleton for URECAMatch.

Tech stack
- Front-end: Next.js, TypeScript, Tailwind CSS
- Back-end: FastAPI, Python 3.12, Pydantic, uv
- Data: Supabase PostgreSQL behind a Store
- Auth and files: Supabase Auth, Supabase Storage
- Hosting: Vercel, Supabase, Vercel Cron
- Notifications: Resend (for email push notifications)
- Testing/CI: pytest, ruff, ESLint, tsc, GitHub Actions

Rules
1. Comments only. Files contain nothing but comments: no imports, functions, classes, types,
variables or SQL. Each file has a one-line header naming its class, then TODOs covering its
responsibility.
2. Folders follow the diagram's stereotypes: «boundary» classes in the front-end, «control»
and «entity» classes in the back-end, and the entity tables in a Supabase schema file.

Output: a folder named ureca-match-app/ (or a downloadable .zip), with a README.md showing
the tree. Confirm at the end that no file contains anything but comments.
```

3.2 Minimal Skeleton Template: Model Response / Results

The AI produced 77 files (zipped separately): one page per boundary class in role-based Next.js route groups, the ApiClient files, 10 controllers, 6 services, 14 entities plus enums, the Store and its two implementations, and 16 test files.

```
urecamatch-skeleton/
├── README.md                4 comment lines
├── api/
│   ├── app/
│   │   ├── main.py
│   │   ├── control/
│   │   │   ├── controllers/   10 controllers
│   │   │   └── services/      6 services
│   │   ├── data_access/    Store + 2 implementations
│   │   └── entity/         14 entities + enums.py
│   └── tests/control/      16 test files
└── web/
    ├── app/                22 pages (one per screen)
    └── lib/api/            3 ApiClient files
```

Fig 1. The skeleton template proposed in the initial AI response

3.3 Minimal Skeleton Template: Discussion

The produced skeleton largely matched the class and boundary diagrams that were provided to the AI in its initial prompt, with

a. `web/app` holding all boundary classes,
b. `api/app/control` holding all controllers and services, and
c. `api/app/entity` storing all entities for URECAMatch.

In line with the given technology stack, the proposed structure (and comments) abstracts data access behind a Store interface with a Supabase implementation. The generated skeleton also created one page for each dialog map screen (22 in total), grouped into role-based Next.js route groups, with each page header named after its screen and the controller it calls. As requested, each generated file includes TODO's or short comments that detailed its responsibilities, and little to no implementation code was present in the generated files.

As such, the team generally agreed with the general organization of the classes, but we opted to rename or reorganize several components to better reflect the standard naming conventions in applications developed with FastAPI, as well as to clearly represent their role in the framework (eg. `control/controllers` to `routers/`, `entity/` to `models/`, `data_access/` to `db/`)

Interestingly, the output response by the AI also included several skeletons of features that were not requested. For example, it added a skeleton mock Store under `data_access/`, as well as a mock API client. These were architectural decisions which the team accepted into the implementation as they greatly simplified the initial testing of the URECAMatch prototype (forgoing the need to set up live API endpoints in its early stages). Other minor additions included the extension of several entities with attributes that our class diagram initially lacked. For example, it added cover images for project listings, as well as justification fields for manual assignments, both of which the team found relevant and decided to keep them, updating the class diagram accordingly.

In general, the skeleton produced by the AI was largely cohesive and did not miss any obvious logical components. However, the team opted to adjust several components of the skeleton for various reasons as discussed above. The final skeleton template that the team adopted as a starting point can be seen in Fig. 2 below.

```
URECAMatch/
├── README.md
├── .github/workflows/ci.yml   checks on every pull request
└── ureca-match-app/
    ├── api/
    │   ├── app/
    │   │   ├── main.py        entry point / driver
    │   │   ├── deps.py
    │   │   ├── errors.py      error functions/formatting
    │   │   ├── settings.py    configuration
    │   │   ├── routers/       10 routers (control)
    │   │   ├── services/      6 services (control)
    │   │   ├── models/        9 enums + 14 models (entity)
    │   │   └── db/            Store, seed data, export
    │   ├── api/index.py       Vercel entry point
    │   ├── tests/             28 test modules by requirement
    │   ├── pyproject.toml     Python dependencies
    │   ├── vercel.json        hosting and daily cron
    │   └── .env.example       environment settings
    ├── shared/openapi.json    generated API contract
    ├── supabase/              schema
    └── web/
        ├── app/               22 pages + 5 layouts (boundary)
        ├── components/        app shell, UI library, listing form
        ├── lib/
        │   ├── api/           ApiClient, live client, types
        │   ├── mock/          mock client
        │   ├── session.ts     session expiry
        │   └── use-api.ts     data hook for pages
        ├── e2e/               end-to-end test
        ├── package.json       web dependencies
        ├── next.config.ts     Next.js settings
        └── .env.example       environment settings
```

Fig 2. The final starting skeleton template for URECAMatch (after hand-modifications by the team)

4. Conclusion

In this supplementary report, two aids for prototyping URECAMatch were generated using Claude Opus 5.5 through the Claude Code agent in VS Code: **i)** an updated technology stack and, **ii)** a minimal code skeleton template. The updated technology stack from the first section of this report was then used as part of the input prompt for the latter, generating an initial skeleton system architecture for URECAMatch (Fig. 1). After manual review and adjustments by the team, the resulting skeleton (Fig. 2) was then obtained as the starting point for the implementation of URECAMatch.